

PhishGuard Project Report

FutureTech HackFest 2026 · Theme: *Innovating Tomorrow with Emerging Technologies* Focus areas: **Applied AI · Cybersecurity** Author: Chiara De Venuto (individual participation)

1. Executive Summary

PhishGuard is an AI email phishing detector that works fully offline. It combines a **machine learning classifier** (TF-IDF text representation with logistic regression) and **15 interpretable heuristic features** to produce a 0-100 phishing risk score, a verdict (SAFE / SUSPICIOUS / PHISHING) and, crucially, **evidence**: the exact features and text signals that drove the prediction. Unlike black-box commercial filters, PhishGuard explains *why* an email is dangerous, which makes it useful for education, awareness campaigns and small organizations that want to avoid cloud dependencies.

Key results: 98.61% holdout accuracy, 98.75% 5-fold cross-validation accuracy, 0 false positives and 2 false negatives on a balanced 720-email curated dataset.

2. Problem

Phishing is still one of the most effective ways for attackers to get in. APWG consistently reports record phishing volumes, with millions of unique phishing sites every quarter and social engineering aimed at people more than at infrastructure. For an individual user or a small organization the question is rarely "is this link malicious?" (blacklists already answer that). The real question is "**is this email trustworthy?**", and the answer is needed *before* the click.

Two practical gaps motivated this project: 1. **Explainability.** Most detectors return a verdict without any reason, so users never learn and cannot defend themselves elsewhere. 2. **Dependency.** Commercial solutions are cloud-based. An email sent to a device with no connectivity, or one that is privacy-sensitive, cannot be analyzed. PhishGuard runs 100% locally.

3. Proposed Solution

email text → TF-IDF (1-2 grams, 1677 features) → hstack → Logistic Regression
→ score 0-100 → 15 heuristic features (scaled) → | verdict +
evidence ←

- **Web UI** (Flask): paste an email, get a score bar, a verdict badge, advice, the active heuristic features and the top contributing text indicators.
- **JSON API** (POST /api/analyze): programmatic use, ready for integration.

Why this approach

- **No external API keys, no cloud, no data exfiltration.** The model is a small local artifact (~300 KB). This is a privacy-first design by choice.
- **Hybrid signals.** TF-IDF captures subtle textual patterns, while heuristics encode expert knowledge (typosquatting TLDs, raw-IP URLs, Reply-To mismatches, urgency and financial trigger words) that ML on small data may miss.
- **Interpretable by construction.** Logistic regression coefficients are directly readable, and the UI surfaces them.

4. Methodology

4.1 Dataset

Balanced, labeled synthetic corpus of **720 emails (360 phishing / 360 legit)**, generated deterministically (seed 42) from documented phishing patterns: typosquatted domains, raw-IP URLs, urgent and financial trigger phrases, generic greetings, Reply-To mismatches, malicious attachments and ALL-CAPS manipulation. It also includes **hard examples** that make the task non-trivial:

- legitimate emails that contain phishing-like words ("verify your email", "password changed", "security settings updated") with legitimate domains;
- low-intensity phishing with a calm tone and no exclamation marks.

No real user data was used. The generator is part of the repository (`phishguard/build_dataset.py`), so the dataset is fully reproducible.

4.2 Features

Text: TF-IDF, n-grams 1-2, 4000 max features (1677 after `min_df=2`), sublinear TF.

Heuristics (15): `num_urls`, `has_ip_url`, `n_click_here`, `n_suspicious_tld`, `urgent_keywords`, `financial_keywords`, `generic_greeting`, `n_exclamations`, `n_all_caps_words`, `caps_ratio`, `body_length`, `n_attachments`, `has_attachment_word`, `reply_to_mismatch`, `has_phone`.

4.3 Model

Logistic Regression (`C=1.0`, `class_weight=balanced`, 3000 iterations) on the stacked sparse matrix. Logistic regression was chosen deliberately: it is a strong baseline for text classification, fast to train and evaluate on modest hardware, and **fully interpretable**.

4.4 Evaluation

- Holdout: stratified 80/20, seed 42. **98.61% accuracy, 100% precision, 97.22% recall, F1 98.59%**. Confusion matrix `[[72, 0], [2, 70]]`, meaning 0 false positives and 2 false negatives.
- **5-fold CV** (mean): 98.75% accuracy, F1 98.73%.
- **Ablation:** text-only and heuristics-only both reach about 99% CV accuracy on this corpus; the combined pipeline is the production choice, and the heuristics supply the evidence layer that the UI shows.

5. Innovation & Uniqueness

1. **Explainability is a feature, not an add-on.** Every verdict comes with the list of active heuristic signals and the top text contributors, so the user learns *what to look for*. That directly supports security awareness, a recognized first line of defense.
2. **Offline-first, privacy-preserving detection.** No email content ever leaves the device. That is unusual among modern detectors.
3. **Reproducible synthetic corpus.** The entire dataset is generated by committed code, giving a transparent and auditable training pipeline.
4. **Dual-layer defense.** ML plus expert heuristics, with both layers exposed.

6. Limitations & Future Work

- The corpus is synthetic, so generalization to real-world mail (Enron corpora, live phishing feeds) still has to be validated. Public labeled datasets and manual triage are the immediate next step.

- Single-token text indicators are rough explanations; a follow-up will generate natural-language rationales (offline LLM or template-based).
- Logistic regression is a strong baseline, not a frontier model. A fine-tuned transformer could improve recall on adversarial examples, at the cost of size and interpretability.
- Roadmap: browser extension, MCP/server integration, a threat-intel feed (blacklist) as a third layer, and better explainability for the SUSPICIOUS band.

7. Tech Stack

Layer	Technology
ML	scikit-learn 1.9 (TF-IDF, LogisticRegression), joblib, scipy
Web	Python 3.12, Flask 3.1
Tests	stdlib unittest (6 end-to-end tests)
Assets	pip requirements only, no external services

8. How to Run

```
bash python3 -m venv .venv .venv/bin/pip install -r
requirements.txt .venv/bin/python -m phishguard.build_dataset #
optional: rebuild dataset .venv/bin/python -m phishguard.model #
optional: retrain .venv/bin/python -m unittest tests.test_app #
tests .venv/bin/python app.py # http://127.0.0.1:5001
```

9. AI Tools Disclosure (transparency compliance)

9.1 Statement of use

This project was developed with the assistance of generative AI tools. AI-assisted coding was used for scaffolding, code generation support, debugging and documentation drafting. The tools were used exclusively as a development aid: the final code was reviewed, executed and tested locally by the author, and every metric reported in this document was produced by running the code itself, not by the tools.

9.2 Legal framework

This disclosure is made in accordance with the following instruments:

(a) **Regulation (EU) 2024/1689 (Artificial Intelligence Act), Article 50** (transparency obligations for providers and deployers of AI systems), applicable since 2 August 2026. PhishGuard is distributed as a research/educational open-source project and does not operate as a provider or deployer of an AI system within the meaning of the Regulation. This disclosure is nonetheless made voluntarily, in line with the transparency principles of Article 50, towards evaluators, users and any third party.

(b) **Italian Law No. 132 of 23 September 2025 (in force since 10 October 2025), Article 13**, which requires that information on the AI systems used be communicated to the recipient of the service in clear, simple and exhaustive language. This disclosure adopts that standard.

(c) **Code of Practice on Transparency of AI-Generated Content** (European Commission, AI Office; final version published 10 June 2026). Adherence is voluntary; this project aligns with the principles of the Code.

9.3 Data and training transparency

- The training dataset is entirely synthetic and self-generated (see Section 4.1 and `phishguard/build_dataset.py`). No third-party copyrighted works, personal data or proprietary data were used for training.
- The application does not collect, store or transmit personal data: analysis is performed fully offline on the user's device. This is consistent with the principles of data minimisation and confidentiality of Regulation (EU) 2016/679 (GDPR).

9.4 Purpose

The purpose of this disclosure is full transparency towards evaluators, users and any third party reviewing this submission.

10. Declaration

This project is original work, developed for the FutureTech HackFest 2026 submission. It uses only open-source libraries. The dataset is generated by code committed to the repository.